



รายงาน PS02: Process Creation & Pipes

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11)
Kernel	6.6.87.2-microsoft-standard-WSL2 (ตรวจด้วย <code>uname -r</code>)
คอมไพเลอร์	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
คำสั่งคอมไพล์	<code>gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c</code> <code>gcc -Wall -Wextra -o q1_2 q1_2_wait.c</code>
ผลการคอมไพล์	ผ่านทั้งสองไฟล์ ไม่มี warning

หมายเหตุเรื่อง WSL2 WSL2 รัน Linux kernel จริง ดังนั้น `fork()`, `wait()`, `pipe()` และคำสั่ง `ps` ทำงานได้ตามปกติ อย่างไรก็ตามตัวเลขขีดจำกัดจำนวน process ในข้อ 3 อาจต่างจาก Linux ที่ติดตั้งแบบ native หรือแบบ VM เพราะ WSL2 จัดสรรหน่วยความจำให้ VM แบบไดนามิก

สรุปความคืบหน้าของรายงานฉบับนี้

ข้อ	หัวข้อ	โค้ด	ผลการทดลอง
1	fork() แบบมีและไม่มี wait()	เสร็จ	เสร็จ
2	Process status และ sleep() (zombie / orphan)	ยังไม่ได้ทำ	ยังไม่ได้ทำ
3	จำนวน process สูงสุดที่ fork() ได้	ยังไม่ได้ทำ	ยังไม่ได้ทำ
4	การสื่อสารผ่าน pipe()	ยังไม่ได้ทำ	ยังไม่ได้ทำ
5	กรณีพิเศษของ pipe (5.1–5.3)	ยังไม่ได้ทำ	ยังไม่ได้ทำ

หมายเหตุ ส่วนที่ยังไม่ได้ทำถูกทำเครื่องหมายไว้ด้วยข้อความสีแดง [xxxxxxxxxx: ...] ในเนื่อรายงาน เพื่อให้หาจุดที่ต้องเติมได้ง่าย ก่อนส่งจริงต้องเติมให้ครบทุกจุดและลบคำสั่ง \todo ออก

1 fork() แบบมีและไม่มี wait()

1.1 Source code

1.1.1 1.1 โปรแกรม helloworld (ไม่มี wait())

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     printf("Hello World!\n");
8
9     pid_t pid = fork();
10    if (pid < 0) {
11        perror("fork");
12        exit(EXIT_FAILURE);
13    }
14
15    if (pid == 0) {
16        /* child */
17        printf("I am the child, my PID = %d, my parent PID = %d\n",
18              getpid(), getppid());
19    } else {
20        /* parent */
21        printf("I am the parent, my PID = %d, my child PID = %d\n",
22              getpid(), pid);
23    }
24
25    printf("Common line printed by PID %d\n", getpid());
26    return 0;
27 }
```

Code 1: q1_1_helloworld.c – fork() without wait()

คำอธิบายการทำงาน: เรียก `fork()` เพื่อสร้าง child process หากเป็น child (`pid == 0`) จะพิมพ์ PID ของตนเองและ parent ส่วน parent จะพิมพ์ PID ของตนเองและลูก เนื่องจากไม่มี `wait()` parent จึงจบการทำงานโดยไม่รอ child

1.1.2 1.2 โปรแกรมที่เพิ่ม wait()

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     printf("Hello World!\n");
9
10    pid_t pid = fork();
11    if (pid < 0) {
12        perror("fork");
13        exit(EXIT_FAILURE);
14    }
15
16    if (pid == 0) {
17        /* child */
18        printf("I am the child, my PID = %d, my parent PID = %d\n",
19              getpid(), getppid());
20    } else {
21        /* parent */
22        wait(NULL);
23        printf("I am the parent, my PID = %d, my child PID = %d\n",
24              getpid(), pid);
25    }
26
27    printf("Common line printed by PID %d\n", getpid());
28    return 0;
29 }
```

Code 2: q1_2_wait.c – fork() with wait()

คำอธิบายการทำงาน: เพิ่มการเรียก wait(NULL) ในฝั่ง parent เพื่อสั่งให้ parent หยุดรอ (Blocked) จนกว่า child จะทำงานเสร็จและคืนสถานะเรียบร้อย บังคับให้ลำดับการแสดงผลของ child เกิดขึ้นก่อน parent เสมอ

1.2 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c
$ ./q1_1
Hello World!
I am the parent, my PID = 3642, my child PID = 3643
Common line printed by PID 3642
I am the child, my PID = 3643, my parent PID = 3642
Common line printed by PID 3643

$ ./q1_1
Hello World!
I am the parent, my PID = 3644, my child PID = 3645
Common line printed by PID 3644
I am the child, my PID = 3645, my parent PID = 3632    <-- PPID changed (Orphan)!
Common line printed by PID 3645
```

Code 3: ผลลัพธ์ของข้อ 1.1 (ไม่มี wait())

```
$ gcc -Wall -Wextra -o q1_2 q1_2_wait.c
$ ./q1_2
Hello World!
I am the child, my PID = 3649, my parent PID = 3648
Common line printed by PID 3649
I am the parent, my PID = 3648, my child PID = 3649
Common line printed by PID 3648
```

Code 4: ผลลัพธ์ของข้อ 1.2 (มี wait())

1.3 อภิปรายผล

จากการทดลองเปรียบเทียบระหว่างโปรแกรมข้อ 1.1 (ไม่มี wait()) และข้อ 1.2 (มี wait()) สรุปความแตกต่างที่สำคัญได้ดังนี้

- **การชิงโครโนซ์และลำดับ Output:** ในข้อ 1.1 ทั้ง Parent และ Child แย่งกันทำงานอย่างอิสระ ลำดับการพิมพ์ข้อความจึงขึ้นอยู่กับ Kernel Scheduler และไม่แน่นอน (Non-deterministic) ในขณะที่ข้อ 1.2 การเรียก wait(NULL) จะทำให้ Parent หยุดรอ (Blocked) จนกว่า Child จะทำงานเสร็จ ลำดับ Output จึงมีความแน่นอน (Deterministic) โดยข้อความของ Child จะปรากฏก่อนเสมอ
- **ปรากฏการณ์ Orphan Process:** ในข้อ 1.1 มีโอกาสที่ Parent จะทำงานเสร็จและจบโปรแกรมไปก่อน ทำให้ Child กลายเป็น Orphan Process และถูกเคอร์เนลยกให้ init (หรือ Subreaper) เป็น Parent ใหม่แทน (สังเกตได้จาก PPID ของ Child ที่เปลี่ยนไปในการรันครั้งที่ 2) ส่วนในข้อ 1.2 จะไม่เกิดเหตุการณ์นี้เนื่องจาก Parent รอ Child อยู่เสมอ

2 Process Status และ sleep()

[ยังไม่ได้ทำ: ยังไม่ได้เขียนโปรแกรมข้อ 2 – ต้องสร้างไฟล์ q2_procstatus.c]

แนวทางที่ต้องทำ เขียนโปรแกรมที่ fork() แล้วใช้ sleep() คุมลำดับการตายของ parent กับ child ให้ได้สองสถานการณ์ตามโจทย์ ระหว่างที่โปรแกรมยังไม่จบ ให้เปิดอีก terminal หนึ่งแล้วรัน `ps -ef | grep <xxxxxxxxxxxx>` เพื่อบันทึกสถานะและ PPID

2.1 Source code

```
1  /* TODO:
2  * 2.1 child dies first -> parent sleeps long, child returns immediately
3  * 2.2 parent dies first -> parent returns immediately, child sleeps long
4  * In both cases the parent must NOT call wait(), so that the
5  * zombie / orphan behaviour becomes visible in ps.
6  */
```

Code 5: q2.c – ควบคุมลำดับการตายด้วย sleep()

คำอธิบายการทำงาน [ยังไม่ได้ทำ: อธิบายว่าวาง sleep() ไว้ตรงไหน เพื่อบังคับให้เกิดสถานการณ์แต่ละแบบ]

2.2 ผลการทดลอง

2.2.1 2.1 Child ตายแล้ว แต่ Parent ยังทำงานอยู่

```
$ ps -ef | grep q2
UID      PID      PPID    C  STIME TTY      TIME    CMD
```

Code 6: ผลของ ps -ef ในกรณีที่ 2.1

PID ของ parent
 PID ของ child
 PPID ของ child
 สถานะของ child ที่เห็นใน ps

2.2.2 2.2 Parent ตายแล้ว แต่ Child ยังทำงานอยู่

```
$ ps -ef | grep q2
```

Code 7: ผลของ ps -ef ในกรณีที่ 2.2

PID ของ parent
 PID ของ child
 PPID ของ child (ก่อน parent ตาย)
 PPID ของ child (หลัง parent ตาย)

2.3 อภิปรายผล

2.3.1 เปรียบเทียบสองสถานการณ์

[ยังไม่ได้ทำ: เขียนอภิปราย] ประเด็นหลักคือกรณี 2.1 จะได้ **zombie process** ส่วนกรณี 2.2 จะได้ **orphan process** ที่ถูก reparent ซึ่งเป็นปรากฏการณ์เดียวกับที่สังเกตได้แล้วโดยบังเอิญในข้อ 1.1 การรันครั้งที่ 2

2.3.2 ปรากฏการณ์ <defunct>

ความหมาย [ยังไม่ได้ทำ: อธิบายว่า <defunct> คือ **zombie process** คือ process ที่จบการทำงานไปแล้ว แต่ยังไม่มี parent มา wait() เก็บ exit status ไป เคอร์เนลจึงต้องเก็บ entry ไว้ในตารางกระบวนการต่อ]

สาเหตุ [ยังไม่ได้ทำ: parent ยังไม่ได้เรียก wait() หรือ waitpid()]

ผลกระทบ [ยังไม่ได้ทำ: ยังกิน entry ในตารางกระบวนการและจอง PID ไว้ ถ้าสะสมมาก ๆ จะทำให้ระบบสร้าง process ใหม่ไม่ได้ ซึ่งเชื่อมโยงกับขีดจำกัดในข้อ 3]

3 จำนวน process สูงสุดที่ fork ได้

[ยังไม่ได้ทำ: ยังไม่ได้เขียนโปรแกรมข้อ 3 – ต้องสร้างไฟล์ q3_forkcount.c]

ข้อควรระวัง ต้องตั้ง `ulimit -u` ก่อนรันทุกครั้ง เช่น `ulimit -u 200` เพื่อไม่ให้ระบบค้าง และควรใช้รูปแบบ recursive ที่ parent `wait()` รอ child เสมอตามที่โจทย์แนะนำ ไม่ใช้การ `fork()` แบบวนลูป ซึ่งจะกลายเป็น fork bomb

3.1 Source code

```
1 /* TODO: recursive form -- parent waits for child, child forks the next
2  * child, until fork() fails. Report the last successful count.
3  */
```

Code 8: q3.c – นับจำนวนครั้งที่ `fork()` สำเร็จ

คำอธิบายการทำงาน [ยังไม่ได้ทำ: อธิบายวิธีนับ กลไกการหยุด และมาตรการป้องกันไม่ให้ระบบค้าง]

3.2 ผลการทดลอง

3.2.1 3.1 การวัดครั้งแรก

```
$ ulimit -u
$ ./q3
```

Code 9: ผลลัพธ์การวัดครั้งแรก

3.2.2 3.2 เมื่อมีโปรแกรมอื่นทำงานค้างอยู่

Code 10: ผลลัพธ์เมื่อรันโปรแกรมอื่นค้างไว้ และหลังปิดโปรแกรมเหล่านั้น

Table 1: เปรียบเทียบจำนวน `fork()` ที่สำเร็จในแต่ละสถานการณ์

สถานการณ์	จำนวน process ที่รันอยู่	<code>fork()</code> สำเร็จ (ครั้ง)
ก่อนรันโปรแกรมเพิ่ม		
ระหว่างรันโปรแกรมเพิ่ม		
หลังปิดโปรแกรมทั้งหมด		

3.3 อภิปรายผล

[ยังไม่ได้ทำ: เขียนอภิปราย] ประเด็นที่ควรกล่าวถึง

- อะไรคือขีดจำกัดที่แท้จริง (`RLIMIT_NPROC` / หน่วยความจำ / ขนาดตารางกระบวนการ)

- `fork()` คืนค่าอะไร และ `errno` เป็นอะไรเมื่อล้มเหลว (EAGAIN หรือ ENOMEM)
- ทำไมตัวเลขจึงเปลี่ยนไปตามจำนวนโปรแกรมที่รันอยู่ และทำไมจึงไม่กลับมาเท่าเดิมเป๊ะ

4 การสื่อสารผ่าน Pipe

[ยังไม่ได้ทำ: ยังไม่ได้เขียนโปรแกรมข้อ 4 – ต้องสร้างไฟล์ q4_pipe.c]

ผลลัพธ์ที่โจทย์กำหนด ต้องได้ output ตามรูปแบบนี้ โดย child เป็นผู้ส่ง และ parent เป็นผู้รับ

```
Child: Child PID: xxxxxx
Parent: Parent PID: yyyyyy
Parent: Hello from child PID: xxxxxx
```

4.1 Source code

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <string.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     int fd[2];
9     /* TODO: pipe(fd) -> fork()
10      *      child : close(fd[0]) then write()
11      *      parent: close(fd[1]) then read()
12      */
13     return 0;
14 }
```

Code 11: q4.c – child เป็นผู้ส่ง parent เป็นผู้รับ

คำอธิบายการทำงาน [ยังไม่ได้ทำ: อธิบายลำดับ pipe() → fork() → ปิดปลายที่ไม่ใช้ → write() / read() และเหตุผลที่ต้องปิดปลายที่ไม่ใช้ เพราะถ้าไม่ปิดฝั่ง write ให้ครบทุก process แล้ว read() จะไม่มีวันได้รับ EOF และจะ block ค้างตลอดไป]

4.2 ผลการทดลอง

```
$ ./q4
```

Code 12: ผลลัพธ์ของข้อ 4

4.3 อภิปรายผล

[ยังไม่ได้ทำ: เขียนอภิปราย] ประเด็นที่ควรกล่าวถึง เช่น ทำไม pipe จึงเป็นการสื่อสารทางเดียว (uni-directional) ทำไม file descriptor จึงถูกสืบทอดไปยัง child ได้ และ read() มีพฤติกรรม blocking อย่างไร

5 การทดลองกรณีพิเศษของ Pipe

[ยังไม่ได้ทำ: ยังไม่ได้ทำการทดลองข้อ 5 ทั้งหมด – ต้องดัดแปลงโค้ดข้อ 4 เป็นสามเวอร์ชัน]

5.1 5.1 ผู้ส่งพยายามอ่านข้อความของตัวเองกลับ

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child ไม่ปิด fd[0] แล้วลอง read() กลับหลังจาก write() เสร็จ]

```
1 /* TODO */
```

Code 13: ส่วนของโค้ดที่แก้ไขในข้อ 5.1

Code 14: ผลลัพธ์ของข้อ 5.1

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายว่าสุดท้ายใครได้ข้อมูลไป เพราะ pipe เป็นคิวแบบ FIFO ที่ข้อมูลถูก *อ่านแล้วหมดไป* ไม่ใช่การ broadcast ผู้อ่านคนแรกที่มาถึงจะดึงข้อมูลออกไปอีกฝ่ายจึงไม่เหลืออะไรให้อ่านและอาจ block ต่าง]

5.2 5.2 ผู้รับอ่านก่อนที่ผู้ส่งจะส่ง

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child sleep() สักครู่ก่อน write() เพื่อบังคับให้ parent เรียก read() ก่อนที่จะมีข้อมูลใน pipe]

```
1 /* TODO */
```

Code 15: ส่วนของโค้ดที่แก้ไขในข้อ 5.2

Code 16: ผลลัพธ์ของข้อ 5.2

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายพฤติกรรม blocking read ว่า read() จะรอจนกว่าจะมีข้อมูล และเงื่อนไขที่ read() จะคืนค่า 0 (EOF) คือเมื่อ write-end ถูกปิดหมดทุก process]

5.3 5.3 ผู้ส่งส่งหลายข้อความก่อนที่ผู้รับจะอ่าน

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child write() หลายครั้งติดกัน แล้ว parent ค่อยเรียก read() ทีหลัง]

```
1 /* TODO */
```

Code 17: ส่วนของโค้ดที่แก้ไขในข้อ 5.3

Code 18: ผลลัพธ์ของข้อ 5.3

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายว่า pipe เป็น byte stream ที่ไม่รักษาขอบเขตของข้อความ (no message boundary) ข้อความหลายชิ้นจึงถูกอ่านรวมกันมา ใน read() ครั้งเดียวได้ และกล่าวถึงขนาด buffer ของ pipe (ปกติ 64 KB บน Linux) ที่ทำให้ write() block เมื่อ buffer เต็ม]

5.4 สรุปผลการทดลองข้อ 5

Table 2: สรุปพฤติกรรมของ pipe ในแต่ละสถานการณ์

สถานการณ์	ผลที่สังเกตได้	คำอธิบาย
5.1 ผู้ส่งอ่านข้อความตัวเอง		
5.2 ผู้รับอ่านก่อนส่ง		
5.3 ส่งหลายข้อความ ก่อนอ่าน		

สรุปผลการทดลอง

สิ่งที่ได้เรียนรู้จากข้อ 1

1. `fork()` คือการเรียกฟังก์ชันเพียงครั้งเดียวที่คืนค่าสองครั้ง เป็นกลไกพื้นฐานที่ทำให้โค้ดชุดเดียวแตกออกเป็นสองเส้นทางการทำงาน
2. ถ้าไม่มีกลไกชิงโครโนส์ ลำดับการทำงานของ parent กับ child เป็นสิ่งที่ **คาดเดาไม่ได้** และขึ้นกับ scheduler ล้วน ๆ การรันโปรแกรมเดิมซ้ำสามครั้ง ให้ผลไม่ซ้ำกันเลย
3. `wait()` เปลี่ยนโปรแกรมจาก non-deterministic เป็น deterministic และเป็นวิธีป้องกัน zombie process ที่ตรงไปตรงมาที่สุด
4. ได้เห็น **orphan process** และการ reparent โดยบังเอิญจากการรันข้อ 1.1 ซึ่งเป็นปรากฏการณ์เดียวกับข้อ 2.2 ต้องการให้ทดลอง
5. `fork()` คัดลอก **ทุกอย่าง** ใน address space รวมถึง stdio buffer ที่ยังไม่ได้ flush ทำให้เกิดข้อความซ้ำเมื่อ redirect output ลงไฟล์ เป็นกับดักที่พบได้บ่อยและแก้ได้ด้วย `fflush(stdout)` ก่อน `fork()`

ปัญหาที่พบและวิธีแก้

- ข้อความ **"HeLlo WorLd!"** ปรากฏสองครั้ง เมื่อ redirect output ลงไฟล์ ตอนแรกเข้าใจผิดว่าโปรแกรมเขียนผิด ภายหลังพบว่าเกิดจาก stdio buffering แก้โดยเพิ่ม `fflush(stdout);` ก่อนเรียก `fork()`
- ผลการรันไม่ซ้ำเดิม ทำให้บันทึกผลได้ยาก จึงต้องรันหลายครั้งและรายงานทุกครั้ง เพื่อแสดงให้เห็นความไม่แน่นอน แทนที่จะรายงานเพียงครั้งเดียว

งานที่ยังเหลือ **[ยังไม่ได้ทำ: ข้อ 2, 3, 4 และ 5 – ต้องเขียนโปรแกรม รันเก็บผล และเขียนอภิปรายให้ครบก่อนส่ง]**

เอกสารอ้างอิง

1. Linux manual pages: `fork(2)`, `wait(2)`, `pipe(2)`, `ps(1)`, `setvbuf(3)`.
2. เอกสารประกอบการสอน CPE 333 Operating Systems, บทที่ว่าด้วย Process Management.